

~ Wind Valley: A Ghibli-Styled Rock, Paper, Scissors Game ~

Project Report — Code Explained Line by Line

Prepared by: Priyam Singh

B.Tech, Computer Science & Engineering (2024–2028)

Dr. K.N. Modi University

Date: September 15, 2026

A desktop GUI game built with Python's Tkinter library, styled with a Studio-Ghibli-inspired pastel art direction.

1. Introduction

This report documents **Wind Valley**, a desktop implementation of the classic Rock, Paper, Scissors game, built entirely in Python using the standard-library **Tkinter** GUI toolkit. Rather than a plain, utilitarian interface, the project explores a soft, pastel, storybook-like visual direction inspired by Studio Ghibli films — hand-drawn-feeling skies, drifting clouds, rolling hills, and warm wooden panels — with the computer opponent playfully named "Totoro".

The goal of this report is to walk through the source file, **ghibli_rps.py**, section by section and line by line, explaining what each part of the code does and why it was written that way.

2. Objective

- Build a fully working Rock, Paper, Scissors game with a graphical interface.
- Practice GUI programming fundamentals using Tkinter (widgets, layout, events).
- Apply custom Canvas drawing to create original background art instead of using images.
- Implement simple animation (cloud drift, result-text pulse) using Tkinter's event loop.
- Manage application state cleanly: scores, round count, and round history.

3. Tools & Technologies Used

Component	Technology / Library
Programming language	Python 3
GUI toolkit	Tkinter (Python standard library)
Graphics	Tkinter Canvas (vector shapes: rectangles, ovals, polygons)
Randomness	Python's built-in random module
Math utilities	Python's built-in math module (used for hill wave curves)
External dependencies	None — runs with a stock Python installation

4. Program Structure Overview

The program is organised as a single class, **GhibliRPS**, which extends **tk.Tk** (the main application window). The class is broadly divided into four groups of methods:

- **Setup** — `__init__`, which configures the window and calls the builder methods.
- **Scenery / drawing helpers** — methods that paint the sky, sun, clouds, and hills onto a Canvas.
- **UI construction** — methods that create and place labels, buttons, and panels.
- **Game logic** — methods that decide the winner, update the score, and reset the game.

5. Detailed Code Explanation

The following sections walk through the source file from top to bottom. Code is shown in shaded boxes, immediately followed by a plain-language explanation.

5.1 Module Docstring and Imports

```
"""
Ghibli-Style Rock, Paper, Scissors
-----
A whimsical, hand-painted-feeling Rock/Paper/Scissors game built with
Python's built-in tkinter library (no extra installs needed).
"""
```

A triple-quoted string at the very top of the file acts as the module's **docstring** — a description of what the file does. It is not executed as code; it exists purely for documentation and shows up if someone runs `help()` on the module.

```
import random
import tkinter as tk
from tkinter import font as tkfont
import math
```

Four modules are imported: **random** is used later to let the computer pick a move unpredictably; **tkinter** (aliased as `tk`) is the GUI toolkit that provides windows, buttons, labels, and the drawing canvas; `tkinter.font` (aliased as `tkfont`) lets the program build custom `Font` objects instead of relying only on defaults; and **math** supplies the `sin()` function used to draw the gentle wave shape of the hills.

5.2 Colour Palette and Game Constants

```
SKY_TOP = "#bfe3f0"
SKY_BOTTOM = "#eaf6e3"
HILL_FAR = "#9ec9a3"
HILL_NEAR = "#6fa876"
HILL_NEAREST = "#4f8f5c"
... (and similar colour constants)
```

A set of module-level constants stores every colour used in the interface as a hexadecimal string (e.g. `"#bfe3f0"` is a pale sky blue). Defining them once at the top — rather than typing the hex codes repeatedly throughout the file — makes the whole palette easy to find and re-tune in one place, which is standard practice for keeping a consistent visual theme.

```
CHOICES = ["Rock", "Paper", "Scissors"]
EMOJI = {"Rock": "\U0001FAA8", "Paper": "\U0001F4DC", "Scissors": "\u2702\uFE0F"}

BEATS = {
    "Rock": "Scissors",
    "Paper": "Rock",
    "Scissors": "Paper",
}
```

`CHOICES` is a list of the three valid moves, used later whenever the program needs to loop through all options or ask `random` to pick one for the computer. `EMOJI` maps each move name to a Unicode emoji character so the interface can show a picture instead of plain text. `BEATS` encodes the rules of the

game as a dictionary: the key is beaten by the value it maps to (Rock is beaten by Paper, is not directly shown here, but is used later to check `BEATS[player_choice] == computer_choice`, meaning "does the player's choice beat the computer's choice?").

5.3 The GhibliRPS Class and `__init__` Method

```
class GhibliRPS(tk.Tk):
    def __init__(self):
        super().__init__()
```

The main application is defined as a class named **GhibliRPS** that inherits from `tk.Tk`, meaning every instance of this class is a Tkinter root window. `super().__init__()` runs Tkinter's own window-initialisation code before any custom setup happens.

```
self.title("Wind Valley – Rock, Paper, Scissors")
self.geometry("880x680")
self.minsize(760, 600)
self.configure(bg=SKY_TOP)
```

These four lines configure the window itself: the title bar text, the initial window size in pixels (880 wide by 680 tall), the smallest size the user is allowed to resize it down to, and the window's own background colour, set to the pale sky-blue constant.

```
self.player_score = 0
self.computer_score = 0
self.round_num = 0
```

Three instance attributes track the game's state: how many rounds the player has won, how many the computer has won, and how many rounds have been played in total. They are initialised to zero when the window is created.

```
self.title_font = self._pick_font(
    ["Papyrus", "Georgia", "Comic Sans MS"], size=30, weight="bold"
)
... (five more similar font definitions)
```

Rather than hard-coding one font family (which might not be installed on every computer), the program calls a helper method, `_pick_font`, for each font it needs, passing a list of preferred font names in order of preference. Six fonts are created this way: a large decorative title font, a header font, a body font, a large font for the emoji move icons, a result-banner font, and a button font.

```
self.bg_canvas = tk.Canvas(self, highlightthickness=0, bd=0)
self.bg_canvas.place(x=0, y=0, relwidth=1, relheight=1)
self.bg_canvas.bind("<Configure>", self._on_resize)
```

A Canvas widget is created to act as the painted background scenery. `highlightthickness=0` and `bd=0` remove the default border Tkinter draws around canvases. The `place()` call stretches the canvas to fill 100% of the window's width and height (`relwidth=1`, `relheight=1`) and pins it to the top-left corner. Finally, `bind("<Configure>", ...)` tells Tkinter to call `_on_resize` automatically every time the window is resized, so the scenery can be redrawn to fit the new dimensions.

```
self._cloud_offsets = [0, 220, 480, 680]
self._build_scene_static = True
```

`_cloud_offsets` stores four starting horizontal positions for clouds, which will be incremented slightly on every animation tick to make them drift across the sky. The boolean flag on the second line is a small note-to-self left in the code (the scenery is always fully redrawn rather than incrementally updated) and does not affect behaviour.

```
self._build_ui()  
self._animate_clouds()
```

The constructor finishes by calling two more methods: `_build_ui()` creates every visible widget (labels, buttons, panels), and `_animate_clouds()` kicks off the recurring animation loop that keeps the clouds moving for as long as the window is open.

5.4 Helper Methods

```
def _pick_font(self, candidates, size, weight="normal"):
    available = set(tkfont.families())
    for name in candidates:
        if name in available:
            return tkfont.Font(family=name, size=size, weight=weight)
    return tkfont.Font(size=size, weight=weight)
```

This method takes a list of preferred font names, checks which fonts are actually installed on the current system (`tkfont.families()` returns that list), and returns the first match it finds as a proper `Font` object. If none of the preferred fonts are available, it falls back to Tkinter's default font family at the requested size and weight, so the program never crashes because of a missing font.

```
def _on_resize(self, event):
    self._draw_scenery(event.width, event.height)
```

This is the callback bound to the canvas's `<Configure>` event mentioned earlier. Tkinter automatically passes an `event` object containing the new width and height, which are forwarded straight into `_draw_scenery` so the background can be redrawn at the correct size.

5.5 Drawing the Scenery

```
def _draw_scenery(self, w, h):
    c = self.bg_canvas
    c.delete("scenery")
```

This is the core painting method. It first grabs a short reference to the canvas and then deletes every shape tagged "scenery" from a previous draw. Every shape created later in this method is given that same tag, so this single call clears the whole picture cleanly before redrawing it — this is what allows the background to redraw smoothly on every resize and animation tick.

```
steps = 40
for i in range(steps):
    t = i / steps
    r1, g1, b1 = self._hex_to_rgb(SKY_TOP)
    r2, g2, b2 = self._hex_to_rgb(SKY_BOTTOM)
    r = int(r1 + (r2 - r1) * t)
    g = int(g1 + (g2 - g1) * t)
    b = int(b1 + (b2 - b1) * t)
    color = f"#{r:02x}{g:02x}{b:02x}"
    y0 = int(h * 0.55 * (i / steps))
    y1 = int(h * 0.55 * ((i + 1) / steps)) + 1
    c.create_rectangle(0, y0, w, y1, fill=color, outline="", tags="scenery")
```

Tkinter's Canvas has no built-in gradient tool, so the sky gradient is approximated by drawing 40 thin horizontal rectangles stacked on top of each other, each one a slightly different colour. For each strip, `t` is a fraction from 0 to 1 representing how far down the gradient that strip sits. The red, green, and blue components of the top and bottom sky colours are linearly interpolated (blended) using that fraction, converted back into a hex colour string, and used to fill a rectangle spanning the strip's vertical slice, which only occupies the top 55% of the window height (`h * 0.55`) so the hills have room below.

```

sx, sy, sr = w * 0.82, h * 0.16, 70
for i in range(5, 0, -1):
    c.create_oval(
        sx - sr * i / 3, sy - sr * i / 3, sx + sr * i / 3, sy + sr * i / 3,
        fill=SUN_GLOW, outline="", stipple="gray25", tags="scenery"
    )
    c.create_oval(sx - 26, sy - 26, sx + 26, sy + 26, fill="#fff9e0",
        outline="", tags="scenery")

```

A soft glowing sun is placed near the top-right of the window (82% across, 16% down). The loop draws five progressively smaller, semi-transparent-looking circles (using a `stipple` pattern, since Tkinter canvases don't support true alpha transparency) from largest to smallest, which builds up a soft glow effect. A final small solid circle is drawn on top as the sun's core.

```

for base_x in self._cloud_offsets:
    self._draw_cloud(c, (base_x % (w + 300)) - 100, h * 0.14, 1.0)
    self._draw_cloud(c, ((base_x + 150) % (w + 300)) - 100, h * 0.24, 0.7)

```

For each of the four stored cloud offsets, two clouds are drawn: a full-size one and a smaller one (scale 0.7) offset a bit further right and lower. The `(w + 300)` modulo operation wraps each cloud's horizontal position back to the left side once it drifts off the right edge, creating a continuous looping drift effect, and subtracting 100 lets clouds start just off-screen.

```

self._draw_hill(c, h, 0.62, HILL_FAR, wobble=0.05)
self._draw_hill(c, h, 0.74, HILL_NEAR, wobble=0.08)
self._draw_hill(c, h, 0.88, HILL_NEAREST, wobble=0.11)

c.tag_lower("scenery")

```

Three overlapping hill shapes are drawn, each starting lower on the screen (0.62, 0.74, and 0.88 as a fraction of the window height) and in a progressively darker green, which creates a simple sense of depth (the near hill is drawn last so it overlaps the farther ones). Finally, `tag_lower("scenery")` pushes every shape tagged `scenery` behind all other canvas items, guaranteeing the painted background always stays beneath the animated foreground elements.

```

def _draw_cloud(self, c, x, y, scale=1.0):
    puffs = [(0, 0, 22), (18, -8, 18), (-18, -6, 16), (34, 2, 14), (-32, 4, 13)]
    for dx, dy, r in puffs:
        r *= scale
        c.create_oval(
            x + dx * scale - r, y + dy * scale - r,
            x + dx * scale + r, y + dy * scale + r,
            fill=CLOUD_WHITE, outline="", tags="scenery"
        )

```

A single cloud is built from five overlapping circles ("puffs"), each defined by an (x-offset, y-offset, radius) tuple relative to the cloud's centre point. Overlapping several circles of different sizes is a simple way to fake the irregular, bumpy silhouette of a cartoon cloud without needing an image file. The `scale` parameter shrinks the whole cloud proportionally, which is how the method produces both the large and small clouds seen in `_draw_scenery`.

```

def _draw_hill(self, c, h, base_frac, color, wobble=0.05):
    w = self.bg_canvas.wininfo_width() or 880
    base_y = h * base_frac
    points = [(-20, h + 20)]
    segs = 10
    for i in range(segs + 1):
        x = -20 + (w + 40) * (i / segs)
        y = base_y + math.sin(i * 1.3 + base_frac * 10) * (h * wobble)
        points.append((x, y))
    points.append((w + 20, h + 20))
    flat = [coord for pt in points for coord in pt]
    c.create_polygon(flat, fill=color, outline="", smooth=True, tags="scenery")

```

This method builds one rolling hill as a filled polygon. It starts and ends with a point far below the visible area on the left and right edges ($h + 20$) so the hill shape always reaches the bottom of the window with no gaps. Between those anchor points, it generates 11 points ($segs + 1$) spaced evenly across the width, and offsets each one's y-coordinate using a sine wave (`math.sin`) so the hill's top edge gently curves up and down rather than being a flat line. The `wobble` parameter controls how pronounced that curve is. Finally the list of (x, y) point tuples is flattened into a single flat list of numbers, because that is the format Tkinter's `create_polygon` expects, and `smooth=True` asks Tkinter to round the corners between points into a smooth curve.

```

@staticmethod
def _hex_to_rgb(hexstr):
    hexstr = hexstr.lstrip("#")
    return tuple(int(hexstr[i:i + 2], 16) for i in (0, 2, 4))

```

A small utility marked `@staticmethod` (meaning it doesn't need access to `self`) that converts a colour string like `"#bfe3f0"` into a tuple of three integers (red, green, blue), each between 0 and 255. It strips the leading `#` and then reads the string two characters at a time as base-16 (hexadecimal) numbers. This is what makes the colour-blending math in the `gradient` and the `darken/lighten` helpers possible.

```

def _animate_clouds(self):
    self._cloud_offsets = [(o + 0.4) for o in self._cloud_offsets]
    w = self.bg_canvas.wininfo_width() or 880
    h = self.bg_canvas.wininfo_height() or 680
    if w > 10 and h > 10:
        self._draw_scenery(w, h)
    self.after(60, self._animate_clouds)

```

This method drives the cloud-drift animation. Each time it runs, it nudges every cloud offset 0.4 pixels to the right, reads the canvas's current width and height (falling back to the initial 880x680 if the widget hasn't been drawn yet), and — as long as the canvas has a sensible size — asks `_draw_scenery` to repaint everything with the clouds in their new positions. The final line, `self.after(60, self._animate_clouds)`, is the key to the whole animation: it tells Tkinter to call this same method again after 60 milliseconds, creating a repeating loop (roughly 16-17 times per second) for as long as the application runs.

5.6 Building the User Interface

```
def _build_ui(self):
    self.title_label = tk.Label(
        self, text="\U0001F343 Wind Valley \U0001F343",
        font=self.title_font, fg=TEXT_DARK, bg=SKY_TOP
    )
    self.title_label.place(relx=0.5, y=46, anchor="center")
```

The interface-building method starts by creating a `Label` widget for the game's title, using a leaf emoji on either side of the text, the decorative title font, dark text colour, and the sky-blue background so it blends with the scenery behind it. `place(relx=0.5, y=46, anchor="center")` positions it horizontally centred (50% across the window) and 46 pixels down from the top.

```
self.subtitle_label = tk.Label(...)
self.subtitle_label.place(relx=0.5, y=82, anchor='center')
```

A smaller subtitle label ("a gentle game of Rock, Paper, Scissors") is placed just below the title using the same centring technique, in the softer body font and a lighter text colour.

```
self.score_frame = tk.Frame(self, bg=PANEL_WOOD,
    highlightbackground=PANEL_WOOD_EDGE,
    highlightthickness=3)
self.score_frame.place(relx=0.5, y=126, anchor='center', width=420, height=54)

self.score_label = tk.Label(
    self.score_frame, text=self._score_text(),
    font=self.header_font, fg=TEXT_DARK, bg=PANEL_WOOD
)
self.score_label.pack(expand=True, fill='both')
```

The scoreboard is built from a `Frame` styled like a warm wooden sign (parchment background colour with a 3-pixel tan border via `highlightbackground/highlightthickness`), sized to 420x54 pixels and centred near the top. Inside that frame sits a `Label` showing the current score text (built by the `_score_text()` helper described later), packed to fill the whole frame.

```
self.stage_frame = tk.Frame(self, bg='')
self.stage_frame.place(relx=0.5, y=270, anchor='center', width=680, height=190)

self.player_card = self._make_card(self.stage_frame, 'You')
self.player_card['outer'].place(x=0, y=0, width=280, height=190)

self.vs_label = tk.Label(self.stage_frame, text='VS', ...)
self.vs_label.place(x=300, y=80, width=80, height=30)

self.computer_card = self._make_card(self.stage_frame, 'Totoro')
self.computer_card['outer'].place(x=400, y=0, width=280, height=190)
```

A wide container frame (the "stage") holds the two battle cards side by side. The player's card is built by the reusable `_make_card` helper (labelled "You") and placed at the left; a "VS" label sits centred between them; and the computer's card, labelled "Totoro", is placed on the right. Using explicit x/y pixel coordinates within the stage frame (rather than percentage-based placement) gives precise control over the layout of this particular section.

```

self.result_label = tk.Label(
    self, text='Choose your move to begin \u2728',
    font=self.result_font, fg=TEXT_DARK, bg=SKY_TOP
)
self.result_label.place(relx=0.5, y=390, anchor='center')

```

A prominent result banner is created with a starting prompt message, using the larger, bold result font. Its text and colour are updated dynamically later, once per round, inside `play_round`.

```

self.buttons_frame = tk.Frame(self, bg=SKY_TOP)
self.buttons_frame.place(relx=0.5, y=460, anchor='center')

self.choice_buttons = {}
colors = {'Rock': ACCENT_RED, 'Paper': ACCENT_BLUE, 'Scissors': ACCENT_GREEN}
for i, choice in enumerate(CHOICES):
    btn = tk.Button(
        self.buttons_frame,
        text=f'{EMOJI[choice]}\n{choice}',
        font=self.button_font, fg='white', bg=colors[choice],
        activebackground=self._darken(colors[choice]),
        activeforeground='white', relief='flat', bd=0,
        width=8, height=3, cursor='hand2',
        command=lambda ch=choice: self.play_round(ch),
    )
    btn.grid(row=0, column=i, padx=14)
    self._add_hover(btn, colors[choice])
    self.choice_buttons[choice] = btn

```

This loop builds the three move buttons (Rock, Paper, Scissors) at once instead of writing near-identical code three times. Each move is given its own accent colour from a small lookup dictionary. For every choice, a flat, borderless `Button` is created showing the move's emoji above its name, coloured with that move's accent colour, and given a hand-shaped mouse cursor. The `command=lambda ch=choice: self.play_round(ch)` line is important: it connects a click on this specific button to the `play_round` method, passing in that button's own choice. The `ch=choice` default-argument trick is required here because, without it, every button's lambda would end up referring to whatever choice happened to equal after the loop finished (a classic Python closure pitfall), rather than to its own individual value. Each button is arranged in a single row using `grid()`, given a hover-colour effect via `_add_hover`, and stored in the `choice_buttons` dictionary for potential later reference.

```

self.history_frame = tk.Frame(self, bg=PANEL_WOOD, ...)
self.history_frame.place(relx=0.5, y=560, anchor='center', width=680, height=90)

self.history_label = tk.Label(
    self.history_frame, text='Round history will appear here \U0001F343',
    font=self.body_font, fg=TEXT_SOFT, bg=PANEL_WOOD,
    justify='left', anchor='w', wraplength=650
)
self.history_label.pack(expand=True, fill='both', padx=14, pady=8)
self.history = []

```

A second wooden panel holds a scrolling-style history of recent rounds. The label inside it is left-justified and word-wraps at 650 pixels so multi-line history text displays neatly. An empty Python list, `self.history`, is created here to store the text of recent rounds as the game is played.

```

self.reset_btn = tk.Button(
    self, text='\U0001F343 New Journey', font=self.body_font,
    fg='white', bg=TIE_BLUE, activebackground=self._darken(TIE_BLUE),
    activeforeground='white', relief='flat', bd=0, cursor='hand2',
    command=self.reset_game
)
self.reset_btn.place(relx=0.5, y=628, anchor='center', width=160, height=34)
self._add_hover(self.reset_btn, TIE_BLUE)

```

A final button, styled like the move buttons but coloured with the muted "tie" blue, lets the player restart the whole game. Its command is wired directly to the `reset_game` method (no lambda is needed here since it takes no extra argument).

5.7 Reusable Card Builder and Small Utilities

```

def _make_card(self, parent, label_text):
    outer = tk.Frame(parent, bg=PANEL_WOOD, highlightbackground=PANEL_WOOD_EDGE,
        highlightthickness=3)
    name_label = tk.Label(outer, text=label_text, font=self.header_font, ...)
    name_label.pack(pady=(10, 0))
    emoji_label = tk.Label(outer, text='\u2753', font=self.big_emoji_font, ...)
    emoji_label.pack(expand=True)
    move_label = tk.Label(outer, text='waiting...', font=self.body_font, ...)
    move_label.pack(pady=(0, 10))
    return {'outer': outer, 'name': name_label, 'emoji': emoji_label, 'move':
        move_label}

```

Since both the player's card and the computer's card share an identical layout — a name at the top, a big emoji in the middle, and a small move label at the bottom — this factory method builds one such card given a parent container and a name to display. It returns a Python dictionary of the card's four widgets, which is stored as `self.player_card` and `self.computer_card`, so later code can reach into a specific piece (for example `self.player_card['emoji']`) and update it without needing separate named attributes for every field of every card.

```

def _add_hover(self, widget, base_color):
    lighter = self._lighten(base_color)
    def on_enter(e):
        widget.configure(bg=lighter)
    def on_leave(e):
        widget.configure(bg=base_color)
    widget.bind('<Enter>', on_enter)
    widget.bind('<Leave>', on_leave)

```

This helper gives any button a subtle interactive hover effect. It pre-computes a lightened version of the button's base colour, then defines two small nested functions and binds them to Tkinter's mouse-enter and mouse-leave events: hovering over the button switches it to the lighter shade, and moving the mouse away restores the original colour. This is what makes buttons feel responsive without needing any image assets.

```

@staticmethod
def _darken(hexstr, factor=0.82):
    r, g, b = GhibliRPS._hex_to_rgb(hexstr)
    r, g, b = int(r * factor), int(g * factor), int(b * factor)
    return f'#{r:02x}{g:02x}{b:02x}'

@staticmethod
def _lighten(hexstr, factor=1.15):
    r, g, b = GhibliRPS._hex_to_rgb(hexstr)
    r = min(255, int(r * factor))
    g = min(255, int(g * factor))
    b = min(255, int(b * factor))
    return f'#{r:02x}{g:02x}{b:02x}'

```

These two static methods generate the pressed/hover colour variants seen throughout the interface. `_darken` multiplies each colour channel by a factor below 1 (making it dimmer, used for the "active"/clicked button state). `_lighten` multiplies each channel by a factor above 1, capping every channel at 255 with `min()` so the result never overflows a valid colour value (used for the mouse-hover state). Both re-use `_hex_to_rgb` to break the colour apart first.

```

def _score_text(self):
    return f'\U0001F343 You: {self.player_score} Round: {self.round_num} Totoro: {self.computer_score} \U0001F343'

```

A small formatting helper that builds the scoreboard's display string from the current values of the three score-tracking attributes. Centralising this string in one method means the scoreboard can be refreshed anywhere in the code with a single call to `self._score_text()`, rather than repeating the same f-string formatting in multiple places.

5.8 Core Game Logic — play_round

```
def play_round(self, player_choice):
    computer_choice = random.choice(CHOICES)
    self.round_num += 1
```

This method runs every time the player clicks one of the three move buttons, receiving that move as `player_choice`. The computer's move is chosen randomly and unpredictably from the same list of options using `random.choice(CHOICES)`, and the round counter is incremented by one.

```
self.player_card['emoji'].configure(text=EMOJI[player_choice])
self.player_card['move'].configure(text=player_choice)
self.computer_card['emoji'].configure(text=EMOJI[computer_choice])
self.computer_card['move'].configure(text=computer_choice)
```

Both battle cards are updated to reveal the moves that were just made: the big question-mark placeholder emoji is replaced with the emoji for the actual move chosen, and the small caption label underneath is updated to the move's name.

```
if player_choice == computer_choice:
    outcome = 'tie'
    result_text = f'A gentle tie – both chose {player_choice}.'
    color = TIE_BLUE
elif BEATS[player_choice] == computer_choice:
    outcome = 'win'
    self.player_score += 1
    result_text = f'You win this round! {player_choice} beats {computer_choice}.'
    \U00001F31F'
    color = WIN_GOLD
else:
    outcome = 'lose'
    self.computer_score += 1
    result_text = f'Totoro wins this round. {computer_choice} beats {player_choice}.'
    \U00001F343'
    color = LOSE_MUTED
```

This is the heart of the game's rules, implemented as a three-way `if / elif / else` chain. If both players picked the same move, it's a tie. Otherwise, the earlier `BEATS` dictionary is consulted: if the move that beats the player's choice equals the computer's move — in other words, if the player's choice beats the computer's — the player wins and their score increases by one. If neither of those conditions is true, the only remaining possibility is that the computer's move beats the player's, so the computer wins and its score increases. Each branch also prepares a descriptive result message and a colour to display it in (gold for a win, a muted red-brown for a loss, and blue for a tie).

```
self.result_label.configure(text=result_text, fg=color)
self.score_label.configure(text=self._score_text())
```

The result banner is updated with the message and colour decided above, and the scoreboard label is refreshed by calling `_score_text()` again so it reflects the freshly updated scores and round number.

```
self.history.insert(0, f'Round {self.round_num}: You {player_choice} vs Totoro {computer_choice} → '
f"{'Tie' if outcome=='tie' else ('You won' if outcome=='win' else 'Totoro won')}")
self.history = self.history[:3]
self.history_label.configure(text='\n'.join(self.history))
```

A one-line summary of the round is built and inserted at the *front* of the history list (`insert(0, ...)`) so the newest round always appears first. Slicing the list with `[ :3]` keeps only the three most recent entries, discarding older ones so the panel doesn't grow forever. The kept lines are then joined with newline characters and written into the history label, so the panel visually shows up to three of the latest rounds, most recent on top.

```
self._pulse(self.result_label)
```

Finally, the result label is passed to a small animation helper that makes it briefly "pulse" in size, giving the round's outcome a bit of visual emphasis instead of just changing silently.

5.9 The Pulse Animation

```
def _pulse(self, widget, step=0):
    sizes = [22, 24, 22, 20]
    if step < len(sizes):
        widget.configure(font=tkfont.Font(
            family=self.result_font.actual('family'), size=sizes[step], weight='bold'))
        self.after(70, lambda: self._pulse(widget, step + 1))
    else:
        widget.configure(font=self.result_font)
```

This method creates a brief pulsing effect purely by cycling the result label's font size through a small sequence — 22, 24, 22, and back to 20 points — with 70 milliseconds between each step, using the same recurring `self.after(...)` technique seen in the cloud animation. The `step` parameter tracks how far through the sequence the animation currently is; once it runs past the end of the `sizes` list, the method resets the label back to its normal, original font so the pulse doesn't leave the text permanently enlarged.

5.10 Resetting the Game

```
def reset_game(self):
    self.player_score = 0
    self.computer_score = 0
    self.round_num = 0
    self.score_label.configure(text=self._score_text())
    self.result_label.configure(text='Choose your move to begin \u2728', fg=TEXT_DARK)
    self.player_card['emoji'].configure(text='\u2753')
    self.player_card['move'].configure(text='waiting...')
    self.computer_card['emoji'].configure(text='\u2753')
    self.computer_card['move'].configure(text='waiting...')
    self.history = []
    self.history_label.configure(text='Round history will appear here \U0001F343')
```

Bound to the "New Journey" button, this method restores the entire game to its starting condition: all three score counters are set back to zero, the scoreboard and result banner are reset to their original text, both battle cards are returned to their placeholder question-mark state, and the history list and label are cleared out. Because every visible piece of state that `play_round` can change is explicitly reset here, calling this method returns the interface to exactly how it looked when the program first launched.

5.11 Program Entry Point

```
if __name__ == '__main__':  
    app = GhibliRPS()  
    app.mainloop()
```

This is the standard Python idiom for marking code that should only run when the file is executed directly (as opposed to being imported as a module elsewhere). It creates one instance of the `GhibliRPS` window — which triggers the entire `__init__` chain described above — and then calls `mainloop()`, which hands control over to Tkinter's event loop. From this point on, Tkinter listens for user actions (button clicks, window resizes) and scheduled callbacks (the cloud animation, the pulse animation) and keeps the window alive and responsive until it is closed.

6. Summary of Features

- Fully playable Rock, Paper, Scissors game with a graphical interface.
- Hand-painted-style background: gradient sky, glowing sun, drifting clouds, and layered rolling hills — all drawn procedurally on a Tkinter Canvas, with no external image files.
- Live-updating scoreboard tracking player wins, computer wins, and round number.
- Animated battle cards that reveal each player's move with emoji icons.
- Rolling history panel showing the three most recent rounds.
- Hover feedback on buttons and a small "pulse" animation on the result text.
- A reset ("New Journey") option that returns the game to its initial state.
- Runs with zero external dependencies — only the Python standard library.

7. How to Run the Project

1. Ensure Python 3 is installed (Tkinter ships with most standard Python installations).
2. Save the source file as `ghibli_rps.py`.
3. Open a terminal in the same folder and run: `python ghibli_rps.py`
4. The game window will open — click Rock, Paper, or Scissors to play a round.

8. Conclusion

This project demonstrates that a simple, well-known game like Rock, Paper, Scissors can become a meaningful learning exercise when combined with a distinctive visual identity. Building the artwork directly from Canvas primitives — rather than importing static images — reinforced core programming concepts such as loops, coordinate math, colour interpolation, and Tkinter's event-driven animation model using `after()`, while the game logic itself provided practice with clean state management and conditional branching. The result is a small, dependency-free desktop application with a cohesive, cozy, Ghibli-inspired look and feel.

— End of Report —